A

How the Advisor system works

A guide to the AI financial chatbot: its parts, how they talk to each other, and where everything lives in the code.

Live: ai-chat-bot-financial-aid.vercel.app
Code: github.com/wnuqbh/AI-Chat-bot-Financial-Aid
October 2026

1 The big picture

The app is split into a **frontend** (what you see in the browser) and a **backend** (the part that thinks, checks permissions and stores data). The backend relies on four outside services.

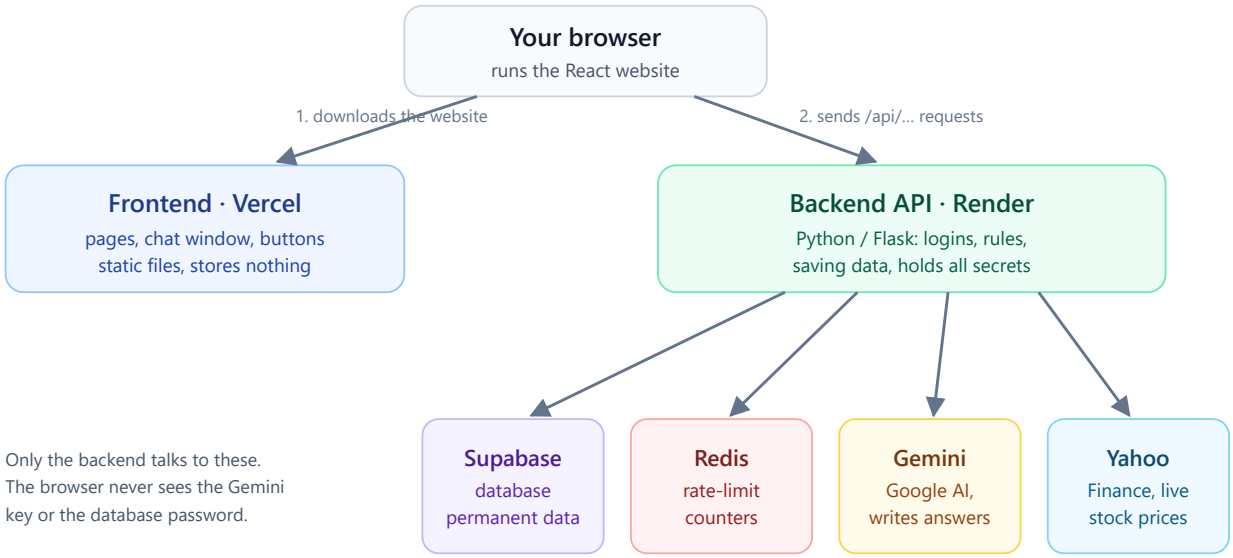


Figure 1. The parts of the system. Frontend and backend are on different domains, so the backend only accepts requests from the Vercel address (this is "CORS").

Frontend
React + TypeScript. Draws the screens and sends requests. Contains no secrets.

Backend
Python + Flask. Checks who you are, enforces the rules, talks to the AI and the database.

Database
Postgres on Supabase. Users, chats, messages, holdings and the audit log.

Redis
A fast counter store used for rate limiting ("10 messages per minute").

Gemini
Google's AI model (gemini-2.5-flash). Writes the advisor's answers.

Yahoo Finance
Current stock prices, cached for 60 seconds.

2 What happens when you send a message

This is the most important flow in the app. It touches almost every part of the system.

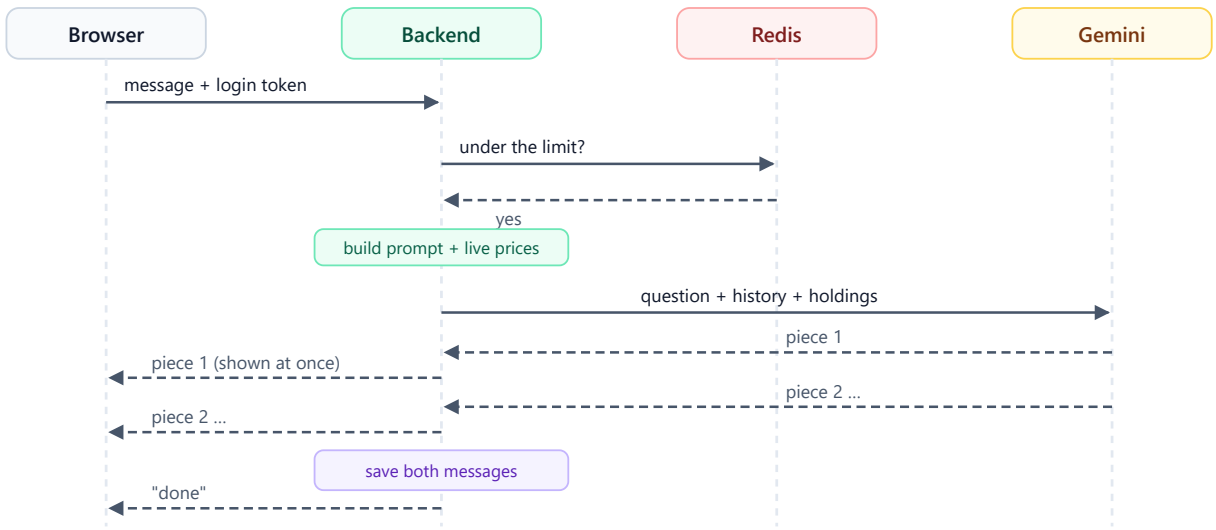


Figure 2. A chat message, step by step. Dashed arrows are replies. The answer is streamed: each piece is shown as soon as Gemini writes it.

1. **You press Enter.** The chat page (`ChatPage.tsx`) shows your message immediately and sends it to `/api/chat/message/stream` with your **login token** attached.
2. **The backend checks you.**
   - Is the token genuine and less than an hour old?
   - Does this conversation belong to you? Users can never read each other's chats.
   - Are you under the rate limit? It checks your counter in Redis.
3. **It builds the question for Gemini** (`advisor.py`): the instructions ("you are a financial education assistant, not a licensed advisor…"), your holdings with **live prices**, the last 20 messages so it remembers the conversation, and your new question.
4. **Gemini answers in pieces.** The backend forwards each piece to your browser straight away, which is why the text appears word by word. This technique is called *streaming*, using Server-Sent Events.
5. **Both messages are saved together.** When the answer is complete, your question and the reply are written to the database in one step, so you never get half a conversation.

> **If Gemini fails**, nothing is saved. Temporary problems (Google overloaded, or the free key's limit of ~10 requests per minute) are retried automatically twice, waiting 1 and then 3 seconds, before you see a message like "temporarily overloaded, try again in a moment".

3 Accounts and security

Signing up and logging in

- Passwords are **hashed** with bcrypt, a one-way scramble. The real password is never stored, so nobody can read it back.
- On login the backend scrambles what you typed and compares. If it matches, you get a **token**: a signed pass valid for 1 hour.
- After **5 wrong passwords** the account locks until an admin unlocks it.

Protections

- **Rate limits:** 10 chat messages per minute and 200 per day per user, and 10 login attempts per minute per IP address.
- **Ownership checks:** every request for a chat or holding is checked against the logged-in user.
- **Audit log:** sign-ups, logins, failed logins and portfolio changes are recorded, and admins can see them.

4 What is stored

The database has seven tables. Chats are nested: a user has **sessions** (the entries in the chat sidebar), each session has a **conversation**, and a conversation holds the **messages**.

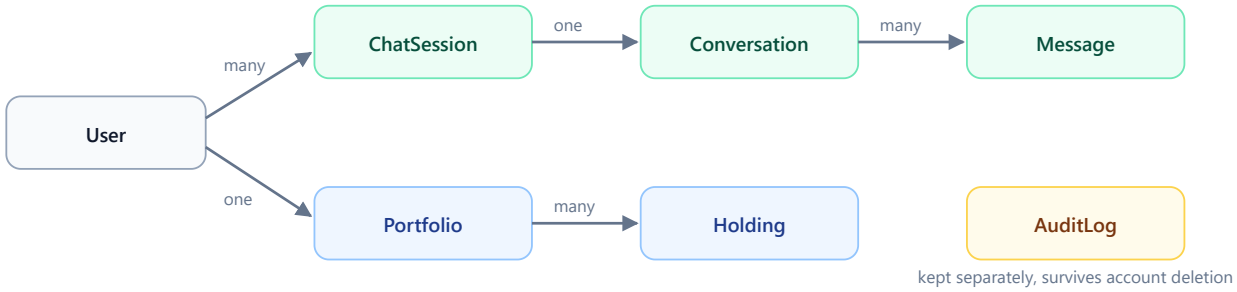


Figure 3. Tables and how they connect. Deleting a user or chat also deletes everything under it, except the audit log.

5 Where things live in the code

```
financial-advisor-chatbot/
├── backend/                      Python (Flask)
│   ├── app/routes/               the URLs: auth.py, chat.py, portfolio.py, admin.py
│   ├── app/services/advisor.py   talks to Gemini: prompt, streaming, retries
│   ├── app/services/quotes.py    gets prices from Yahoo (60-second cache)
│   ├── app/models/               the database tables
│   ├── migrations/               history of database structure changes
│   ├── config.py                 settings: limits, model name, …
│   └── tests/                    37 automatic tests
├── frontend/                     React (TypeScript)
│   ├── src/pages/                Chat, Portfolio, Admin and Login screens
│   ├── src/api.ts                every call to the backend goes through here
│   └── src/portfolio.ts          value, gain and weight calculations
├── docker-compose.yml            run everything locally in containers
├── render.yaml                   backend hosting setup
└── docs/DEPLOYMENT.md            how it is deployed
```

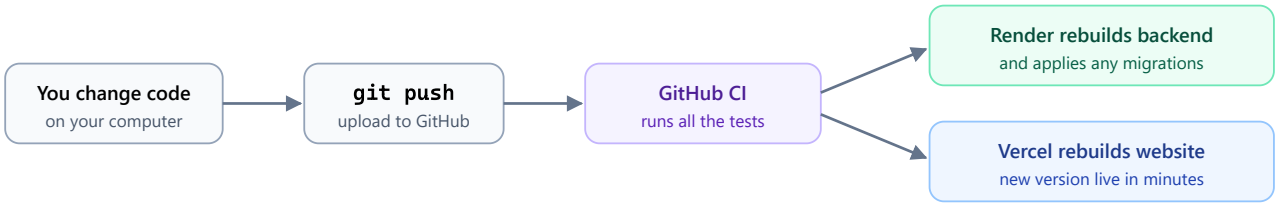
Good places to start reading: `frontend/src/pages/ChatPage.tsx` (what happens on screen), then `backend/app/routes/chat.py` (what the backend does with a message), then `backend/app/services/advisor.py` (how the question reaches Gemini).

6 Three copies of the app

The same code runs in three places. **Each has its own database and its own accounts.** An admin on your computer is not an admin on the live site.

	ADDRESS	DATABASE	HOW TO START IT
Local development	`localhost:5173`	a file: `backend/instance/dev.db`	`python main.py` and `npm run dev`
Docker (local test)	`localhost:8080`	Postgres inside Docker	`docker compose up --build`
Live	`…vercel.app`	Supabase	automatic, on every push

7 How updates go live



If you change the database structure (for example, add a column), a **migration** file describes the change. Render applies it automatically when it starts, without losing existing data.

> **Free-plan note:** Render puts the backend to sleep after 15 minutes without visitors. The first request after that takes about a minute while it wakes up.

8 Glossary

API
The set of URLs (`/api/…`) the frontend uses to ask the backend for things.

Token (JWT)
A signed pass you get when you log in. It proves who you are on every request and expires after 1 hour.

Hashing
A one-way scramble used for passwords, so the original can't be recovered.

Streaming
Sending the answer in pieces as it's written, instead of waiting for all of it.

Rate limit
A cap on how many requests someone can make in a period of time.

CORS
A browser rule: a website may only call a backend on another domain if that backend allows it.

Migration
A file describing a change to the database structure, applied in order.

Docker
Packages each part of the app into a container, so it runs the same everywhere.

CI
Continuous Integration: GitHub runs all the tests automatically on every push.

Environment variable
A setting such as an API key, stored on the host instead of in the code.

For education only. Not financial advice.